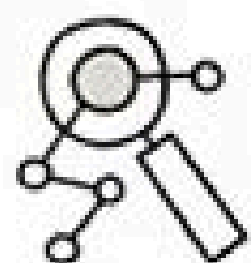


How does Claude Code work?



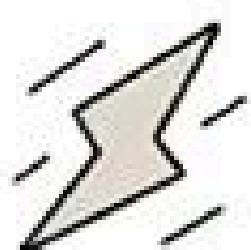
Understands your code

Run agentic, multi-file tasks using deep codebase awareness. No need to manually add files to context—Claude searches your codebase as needed



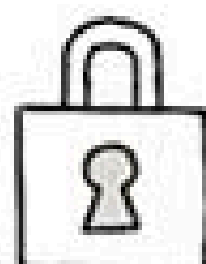
Works with all your tools

Uses all your command line tools the same way you do. Claude Code can test, build, and commit code using tools like git, Sentry, Kubernetes, and more



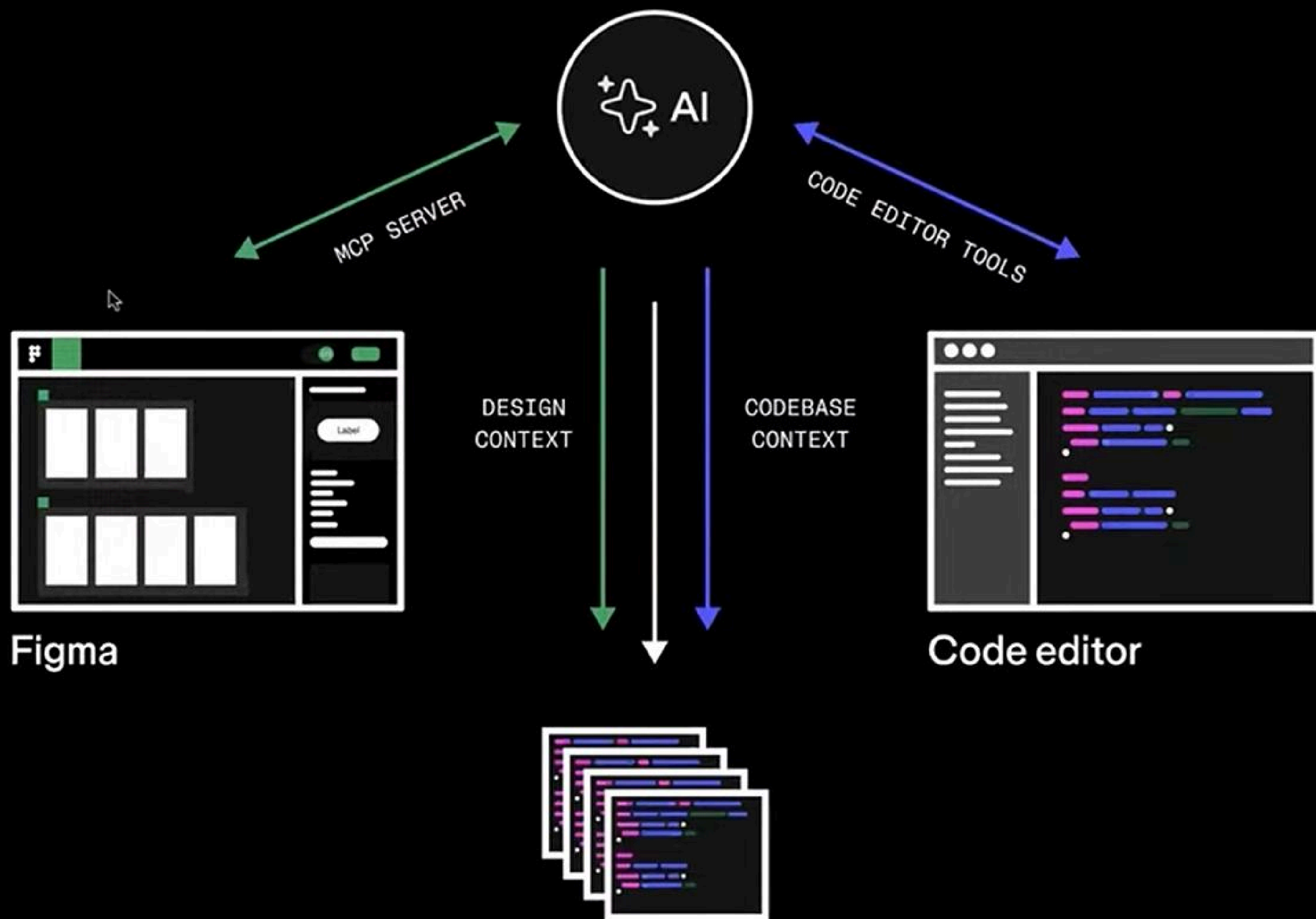
Runs anywhere

Run in your terminal, your IDE, or deploy as a headless CLI tool with the SDK. Claude Code is infinitely flexible for how you prefer to work



Secure by design

Queries go directly to models via API, no intermediary servers or code indexes. Claude Code shows its work and asks user approval before making changes



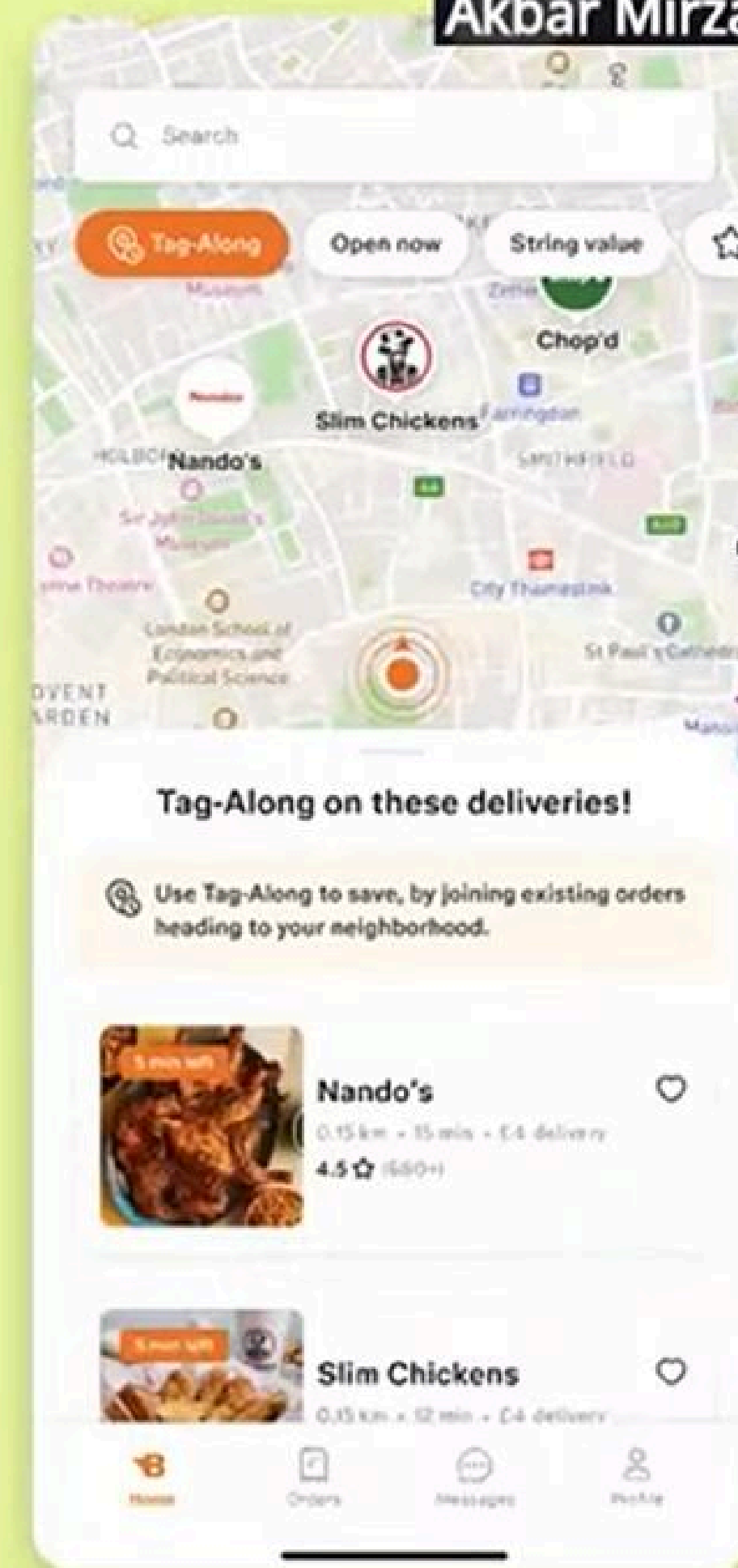
Figma

Code editor

Code Generation
Our code representation of what that



Akbar Mirza (he/him)



End product



DESIGN SYSTEM TEAMS

- Code Connect
- Figma Components
- Variables, Styles, Typography



DEVELOPMENT TEAMS

Dev Mode
MCP server

Gives Figma data
to AI models.



AI coding
assistant

Generates code.

New workflows with Claude Code

0 to 1 explorations



Exploring new features or $0 \rightarrow 1$ ideas in silo

Building in the code



Prototyping new features in context of the full product

Understanding the code base



Design and iterate on features based on how your product works

0 to 1 explorations

Transform new ideas into working prototypes quickly using Figma Dev Mode + Claude Code for perfect iteration via code



meaghan choi



an existing architecture, and that understanding of how the code is really important.

Building in the Code

Direct iteration and testing in the actual codebase rather than isolated design files.

Requirements: GitHub access and company repository permissions

www.youtube.com – to exit full screen, press Esc



Good afternoon, Mea

+ Create a prompt

Generate a prompt

Claude 4 is here
Try Claude Sonnet 4 and Claude Opus 4 today

Untitled
May 20, 2025 by Meaghan Choi

Untitled
May 20, 2025 by Meaghan Choi

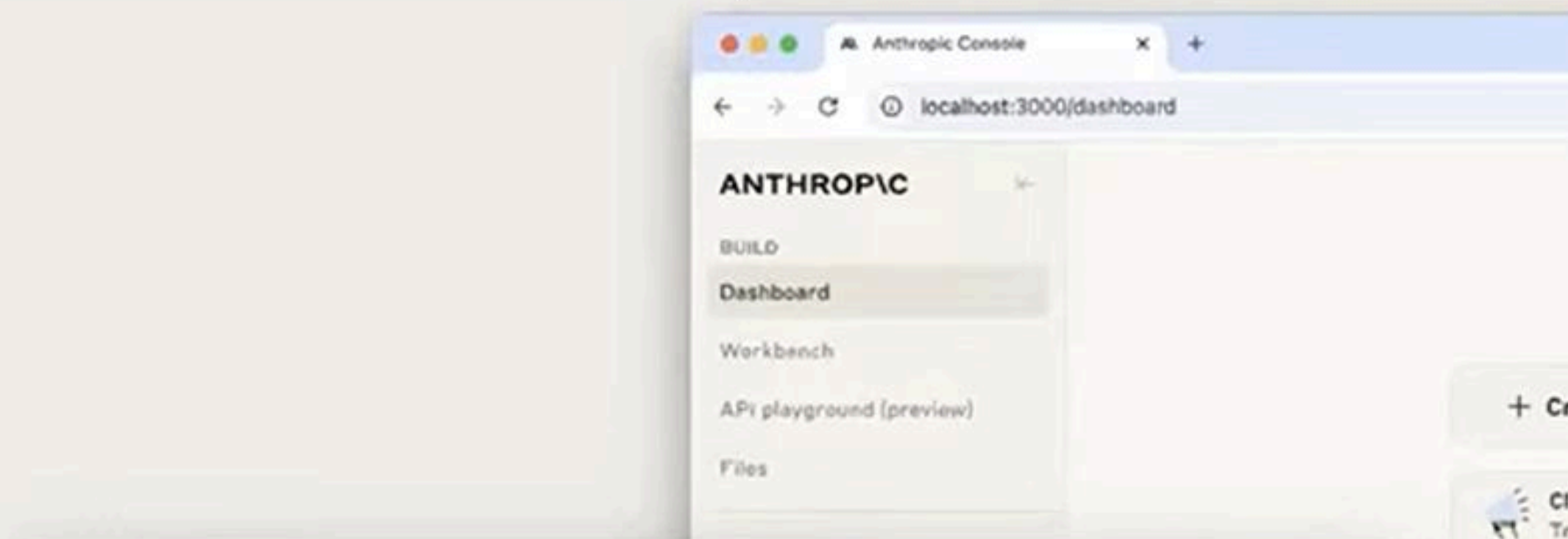
Untitled
May 20, 2025 by Meaghan Choi

Untitled
May 20, 2025 by Meaghan Choi

Untitled
May 20, 2025 by Meaghan Choi

Thinking signature testing

API status Help & support Feedback



```

* Nav Dividers
* Nav Dividers (claude)
next-server | (v15.3.2)

- Remove divider rendering from SectionHeader component
- Remove showTopDivider prop from all SectionHeader usages

• Now I'll remove the dividers from the left navigation. Let me start by working on the first task.

• Update Todos
  - Remove Divider component definition from SidebarNav/index.tsx
  - Remove divider rendering from SectionHeader component
  - Remove showTopDivider prop from all SectionHeader usages

• Update(components/SidebarNav/index.tsx)

Edit file
components/SidebarNav/index.tsx

25   const collapsedWidth = "3rem";
26   const expandedWidth = "16rem";
27
28   const Divider = () => {
29     return (
30       <div className="p-1 w-full">
31         <div className="h-px w-full bg-border-300" />
32       </div>
33     );
34   };
35
28   const SectionHeader = ({
29     concise,
30     children,
31     showTopDivider = false,
32     concise: boolean,
33     children: React.ReactNode;
  
```

Yeah, and I... I love the emphasis of keeping the human in the loop and having those conversations, right

Understanding the code base

Learn how features are implemented to make more informed design recommendations

Requirements: GitHub access and company repository permissions

www.youtube.com — to exit full screen, press `Esc`

meaghan choi

```
claude
* Design Feedback (claude)  ⌘1  esbuild  ⌘2  claude  ⌘3  +

- IDE Status Indicator
- ErrorStatusDisplay (unified errors)
- Permission mode warnings
- Token warning (approaching limit)
- Auto-updater status

This creates a cleaner separation where the left shows "working context" and the right shows "things needing attention".

> General question - how should we deal with conflicting "Modes", technically you can be in debug mode and auto-accept, or verbose mode + plan mode

• Great question! Looking at the current modes, they actually fall into different categories that can coexist:

Input modes (mutually exclusive):
- prompt (default)
- bash (! prefix)
- memory (# prefix)
- memorySelect (after # submit)

Permission modes (mutually exclusive):
- default
- plan (⌘)
- acceptEdits (⌘⌘)

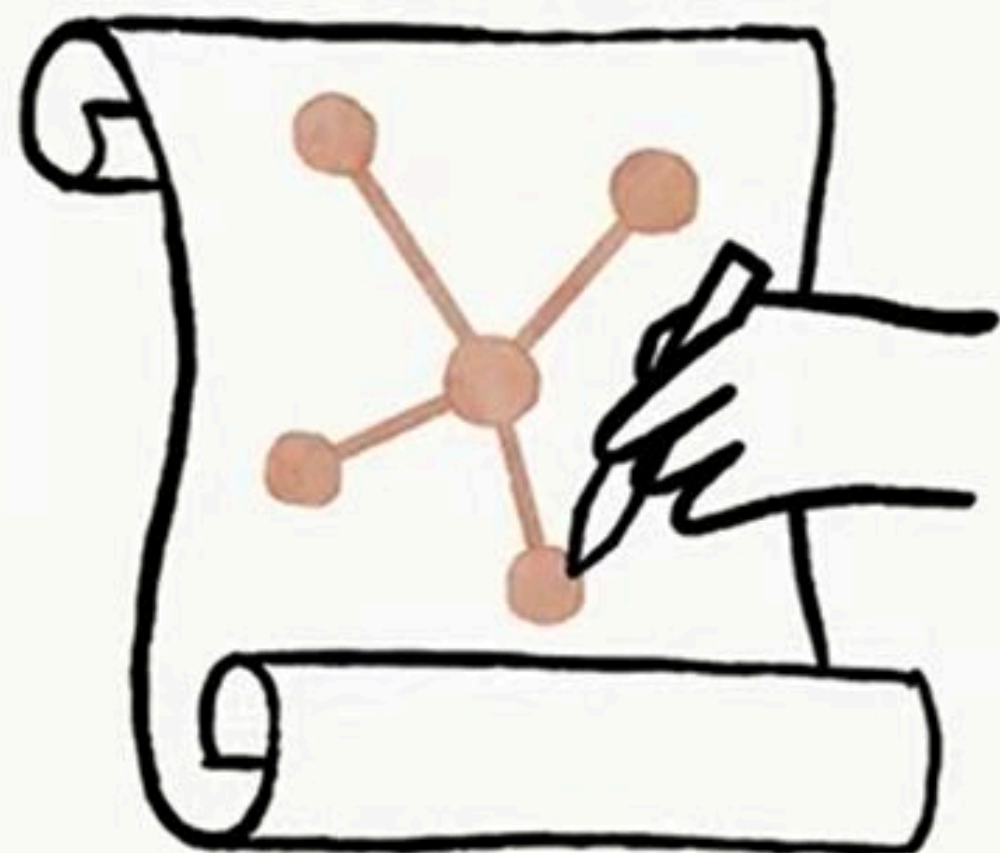
Session flags (can combine):
- debug (--debug flag)
- verbose (ctrl+r toggle)
- vim mode

So you're right - you can have multiple active at once. For the left side display, here are a few approaches:

Option 1: Stack everything (current approach)
-- INSERT -- | ⌘ plan mode on | Debug mode | 12,543 tokens
Problem: Can get very wide/cluttered
```

And generally, I would maybe do this over a week of discussion back and forth, and meeting time with a PM and an engineer, but I was

The CLAUDE.md File



CLAUDE.md

Hierarchical and updatable
instruction management
across repositories and users

User Preferences

- I am a product designer with little experience with coding - NOT a developer
- I need much more detailed explanations than you would give to a senior developer
- Always make smaller, incremental changes rather than large modifications
- I want to learn while coding, so break everything down into simple steps
- For larger or riskier changes, provide specific warnings and signals like "⚠️ LARGE CHANGE ALERT" or "🔴 HIGH RISK MODIFICATION"
- Always remind me to verify larger changes before they're implemented

Be in Learning Mode

- When writing code or concepts, provide educational context and explanations. Break down complex topics into digestible parts, explain your reasoning process, and. Aim to help me understand not just what to do but why it works that way. Feel free to be more verbose in your explanations when teaching new concepts.
- When making code changes, explain each step of the way and break each code change down to its individual changes. Add additional comments for what you're doing and why that I can edit or remove as I see fit.
- Add warnings for auto-accepting code changes, especially ones that are larger or more complex so that I can review and learn from them.
- Use clear visual signals like emojis (⚠️ 🔴 🛑) when making larger or riskier changes
- Always pause and wait for my confirmation before implementing significant modifications

which, at the end of the day, is built for developers to be able to also be built for



Claude Code augments your work as a designer

1 Explore	2 Discover	3 Design	4 Prototype	5 Polish
Rapid prototyping	Explore codebase and history	Plan projects	Implement code & test	The last 2%
Make it real fast	Understand the current state	Identify edge cases and scope	Test new features in your product	Design polish and quality

Claude Code can use you team's real codebase (GitHub/GitLab) & Figma Files

Very niche workflows, but ones that

Source :

<https://www.youtube.com/live/x2LGggL6BNI?si=nyubcckzCgtl9Y2g>